

SGLang: An Efficient Open-Source Framework for Large-Scale LLM Serving

Ke Bao @ LMSYS Org

What is SGLang

sgl-project/**sglang**

SGLang is a fast serving framework for large language models and vision language models.



501 Contributors

494 Issues

230 Discussions

16k Stars

2k Forks



GitHub

GitHub – sgl-project/sglang: SGLang is a fast serving framework for large ...

SGLang is a fast serving framework for large language models and vision language models. – sgl-project/sglang



- **SGLang** is a **fast serving engine** for **LLMs** and **VLMs**.
- Among fully open-source LLM Inference Engines, SGLang currently achieves **SOTA** performance.
- Meanwhile, its **elegant, lightweight and customizable design** has attracted wide adoption from academics, big tech companies and startups. (**xAI, NVIDIA, AMD, Google, Oracle, AWS, Linkedin, Cursor, Baseten, Nebius, Atlas Cloud etc.**)
- In on-policy RLHF, inference engines are crucial for efficient policy model execution, and **SGLang excels as a high-performance solution**.

About SGLang Team

SGLang Team is incubated by [LMSYS Org.](#)

- **Major Maintainers:** Lianmin Zheng, Ying Sheng, Liangsheng Yin, Yineng Zhang, Ke Bao, Byron Hsu, Chenyang Zhao, Zhiqiang Xie, Jingyi Chen, Xiaoyu Zhang, Baizhou Zhang, Yi Zhang, Jiexin Liang, Chang Su, Hai Xiao.
- **Contributors :** [500+](#)

Contributors 518

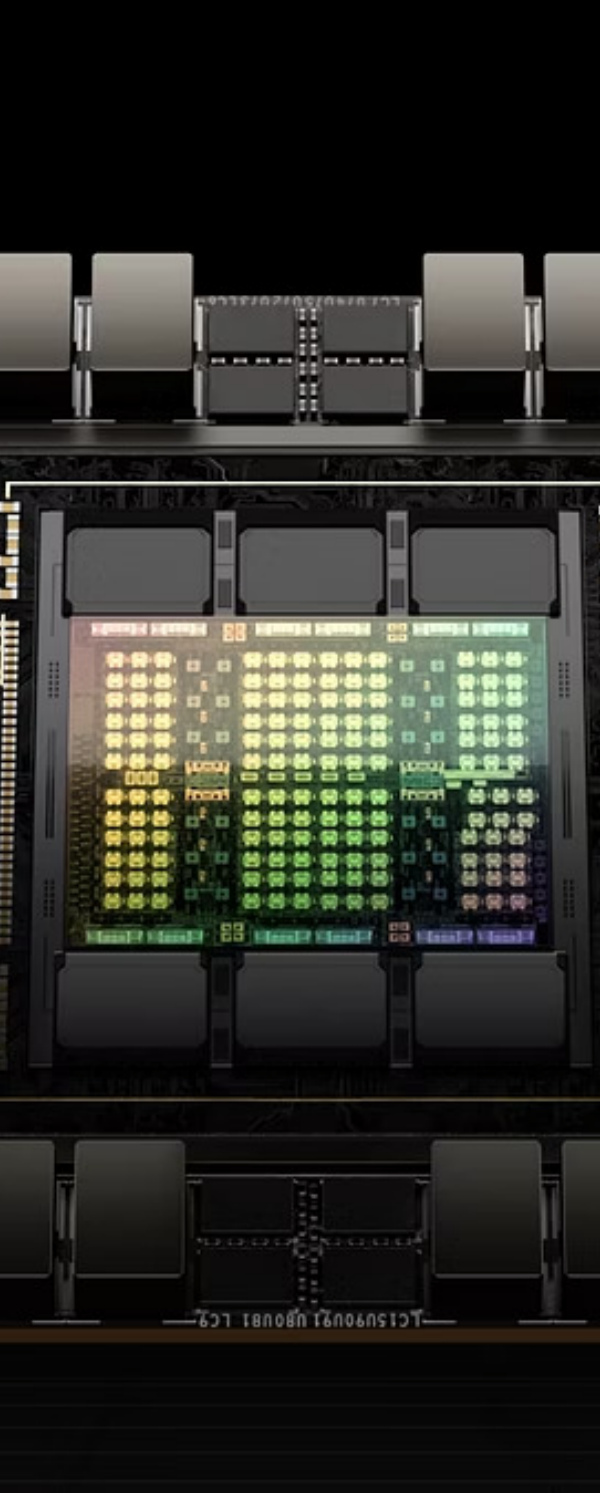


[+ 504 contributors](#)



Remarkable Milestone

- 2023/12-2024/2: [Initial Motivation](#), Structured LM Programming, Prefix Caching and Constrained Decoding
- 2024/7: [Leading Performance](#) among inference engines on Llama3
- 2024/9: [v0.3 Release](#), 7x Faster **DeepSeek MLA**, 1.5x Faster **torch.compile**, Multi-Image/Video LLaVA-OneVision
- 2024/12: [v0.4 Release](#): Zero-Overhead Batch Scheduler, Cache-Aware DP Router, X-Grammar Integration, The First to Serve DeepSeek V3
- 2025/01: SGLang provides **day one support** for DeepSeek V3/R1 models on NVIDIA and AMD GPUs with DeepSeek-specific optimizations. ([10+ companies!](#))
- 2025/05 🔥🔥🔥 First open-source implementation of DeepSeek V3/R1 **expert parallelism** with **prefill-decode disaggregation**. Achieves **52.3K in-tok/s**, **22.3K out-tok/s** on **96 GPUs**—**5x** faster than vanilla TP. ([Blog](#))
- SGLang has seen extensive adoption and serves as the **dominant** inference engine for **AMD** and the **default** inference engine for **xAI**.



Leading Features

1

Efficient KV Cache Reuse with RadixAttention

2

Zero-Overhead Batch Scheduler

3

Speculative Decoding & Constrained Decoding

4

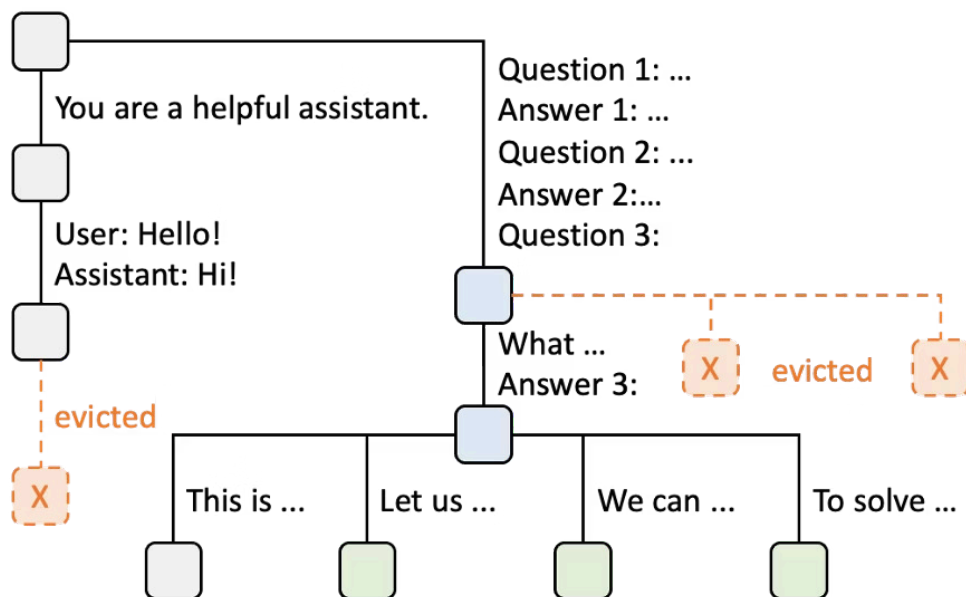
Prefill and Decoding Disaggregation

5

Large-scale Expert Parallelism

Efficient KV Cache Reuse with RadixAttention

RadixAttention for KV Cache Reuse

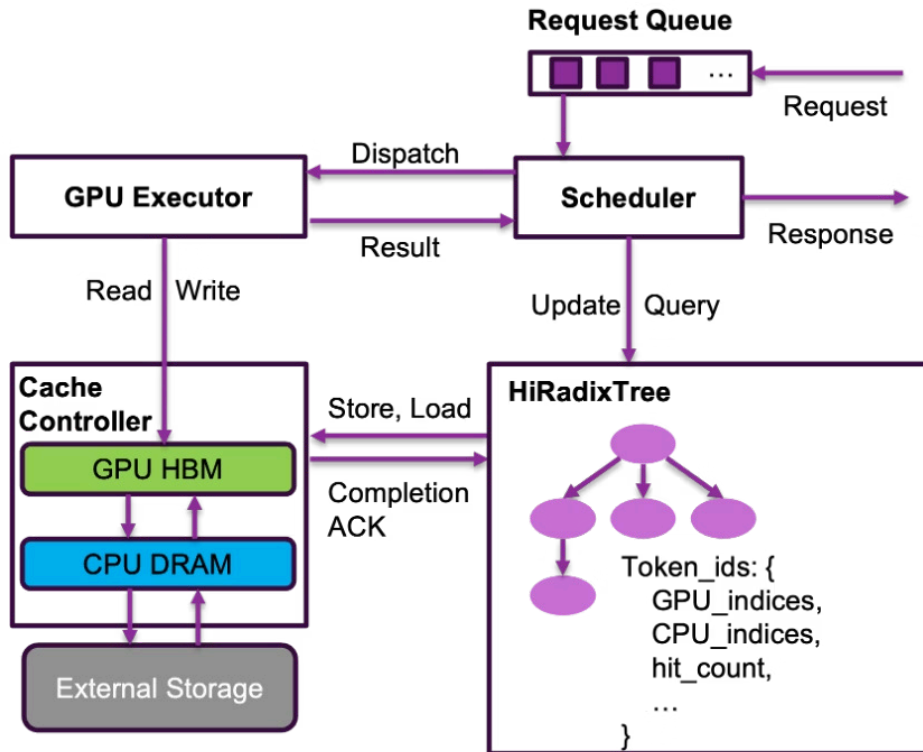


- In real-world LLM applications, many requests often share **identical prompt prefixes**
 - Multi-turn conversations
 - Few-shot prompting
 - Question answering
- **RadixAttention** is designed to **reuse KV cache** across multiple requests with shared prefix.
- **SGLang** maintains an LRU cache of KV cache of all requests in a **Radix Tree**, for efficient **prefix matching, insertion, and eviction**.

For more details:

SGLang: Efficient Execution of Structured Language Model Programs (<https://arxiv.org/pdf/2312.07104>)

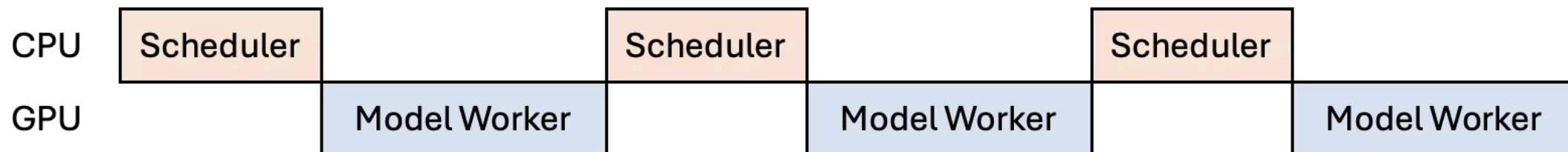
Hierarchical Caching



- **Hierarchical caching** is supported in SGLang to maximize KV cache reuse and reduce redundant computation.
 - It offloads KV caches from **GPU memory** to **CPU memory** when GPU memory is insufficient.
 - We plan to extend this mechanism to support **storage system** level caching as well.
- **Hierarchical caching Design**
 - **Versatile control plane** achieves resource-aware scheduling and effective latency hiding.
 - **Efficient I/O data plane** with specialized CUDA kernels.
 - Ongoing storage layers integration (Generic APIs compatible with popular storage layers).

Zero-Overhead Batch Scheduler

CPU overhead



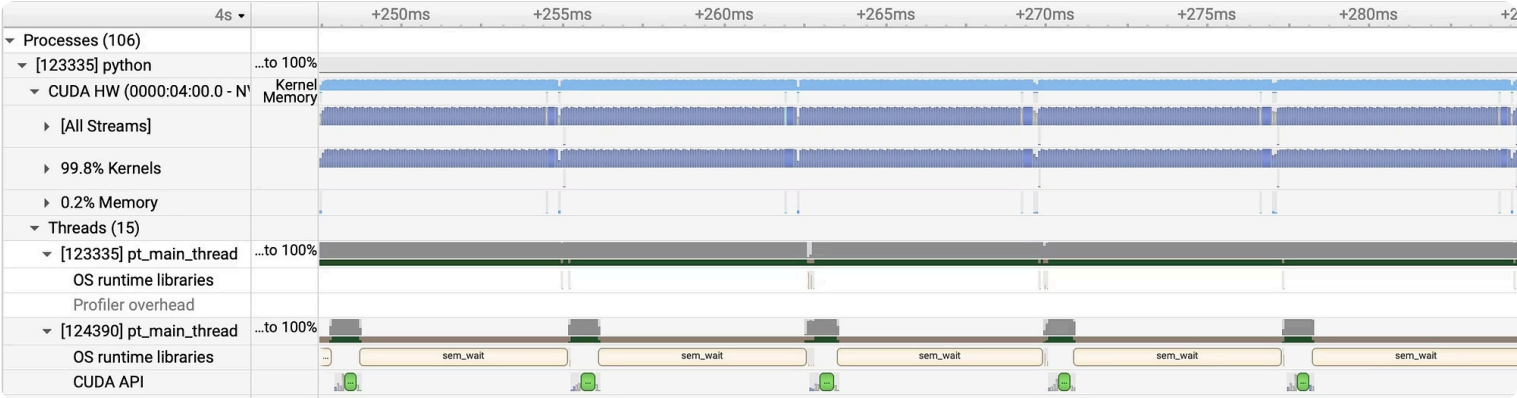
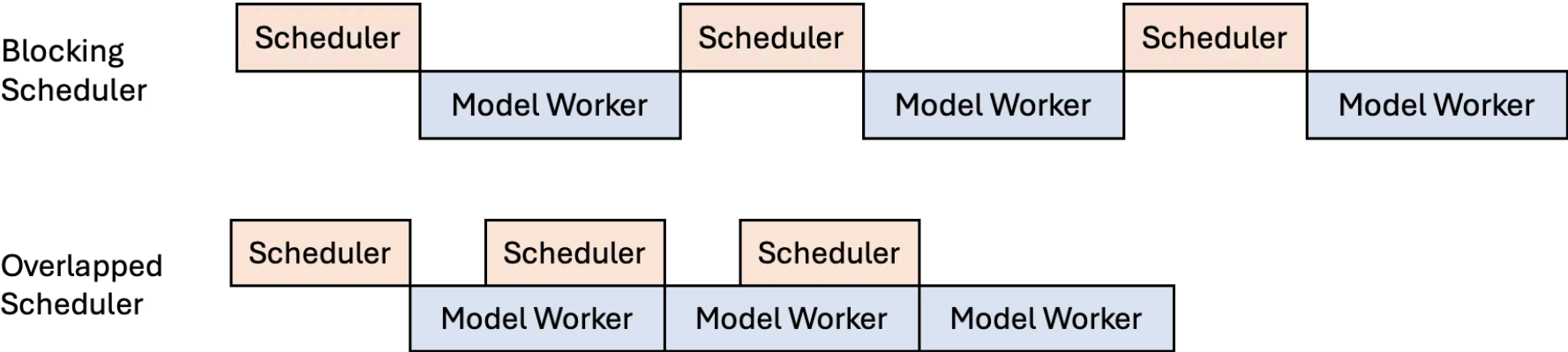
Jobs of the CPU scheduler

- Receives input messages from the user
- Processes results from the model worker
- Checks the stop conditions
- Runs prefix matching and request reorder
- Allocates memory for the next batch

Pseudo code (every line is blocking)

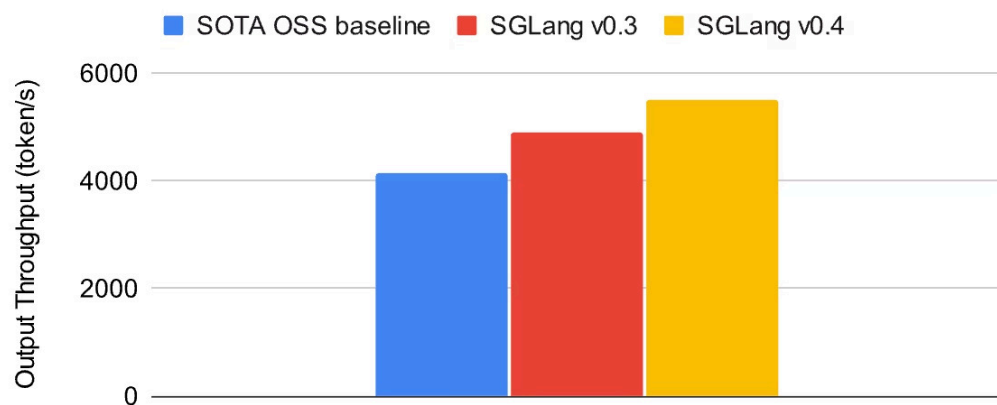
```
while True:
    recv_reqs = recv_requests()
    process_input_requests(recv_reqs)
    batch = get_next_batch_to_run()
    result = run_batch(batch)
    process_batch_result(batch , result)
```

Overlapped scheduler



Benchmark Results

Llama 3.2 3B Throughput Benchmark (Higher is Better)

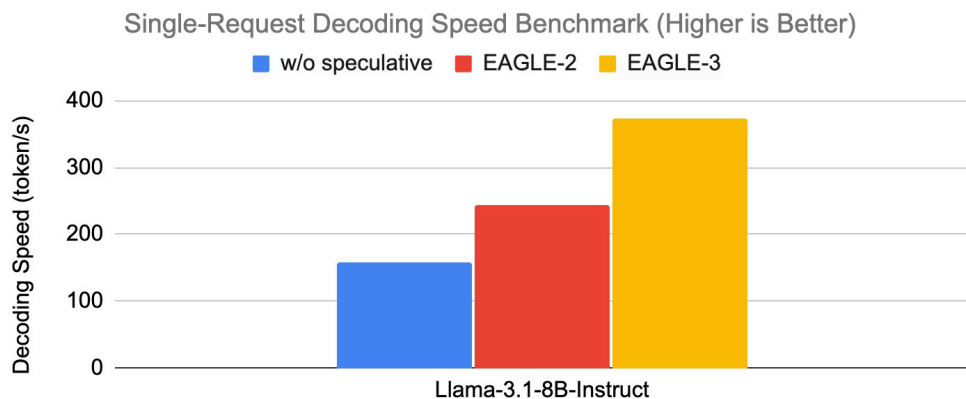


- With this optimization, **SGLang v0.4** achieves a **1.1x** speedup against its previous version.
- The speedup is most significant on small models and large tensor parallelism sizes.

For more details: <https://lmsys.org/blog/2024-12-04-sglang-v0-4/#zero-overhead-batch-scheduler>

Speculative Decoding & Constrained Decoding

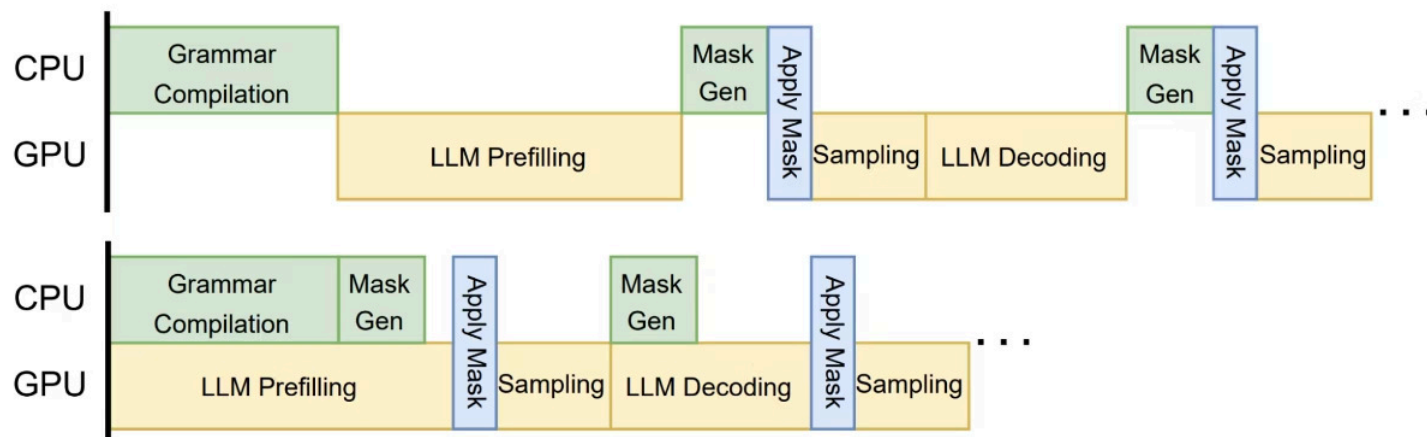
Speculative Decoding with EAGLE 2 & 3



- [EAGLE](#) is a SOTA speculative decoding algorithm.
- In SGLang, both [EAGLE-2](#) & [EAGLE-3](#) speculative decoding are supported. We are the [first](#) to support EAGLE-3 with collaboration with EAGLE team.
- Achieved **1.6x** decoding speedup for EAGLE-2 and **2.4x** decoding speedup for EAGLE-3 on Llama 3.1 8B with single request.

For more details: https://docs.sglang.ai/backend/speculative_decoding.html

Constrained Decoding with XGrammar



- **Zero-Overhead Integration** with XGrammar.
- **10x faster** than other open-source solutions for constraint decoding tasks.

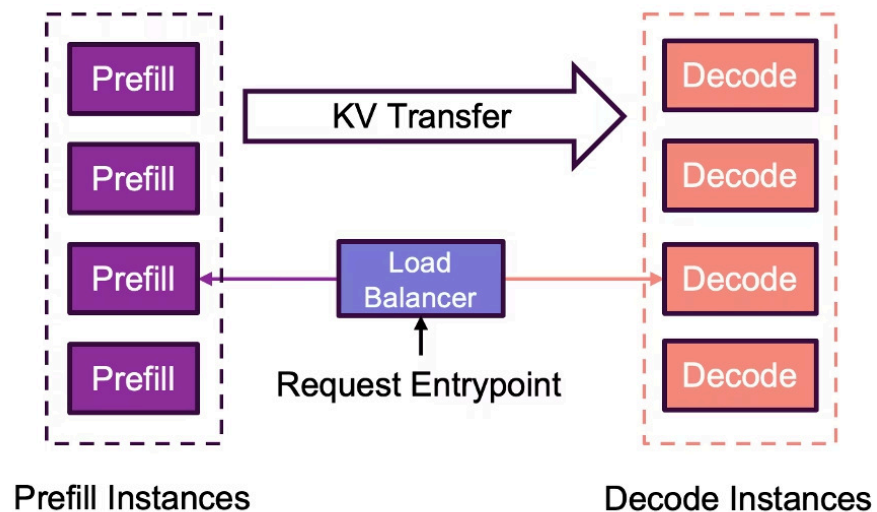
For more details: <https://blog.mlc.ai/2024/11/22/achieving-efficient-flexible-portable-structured-generation-with-xgrammar>

Prefill and Decoding Disaggregation

Why PD Disaggregation

- Prefill and decode exhibit different characteristics:
 - Prefill phase is **computation-intensive**, processing the entire input sequence.
 - Decode phase is **memory-intensive**, reading the KV cache for token generation.
- Issues with Non-Disaggregation
 - **Prefill Interruption**: Incoming prefill batches frequently interrupt ongoing decode batches, causing substantial delays in token generation.
 - **DP Attention Imbalance**: In DP attention, one DP worker may process a prefill batch while another handles a decode batch simultaneously, leading to increased decode latency.
 - **Incompatible with DeepEP**: DeepEP executes different dispatch modes for prefill and decode, making unified scheduling incompatible with DeepEP.

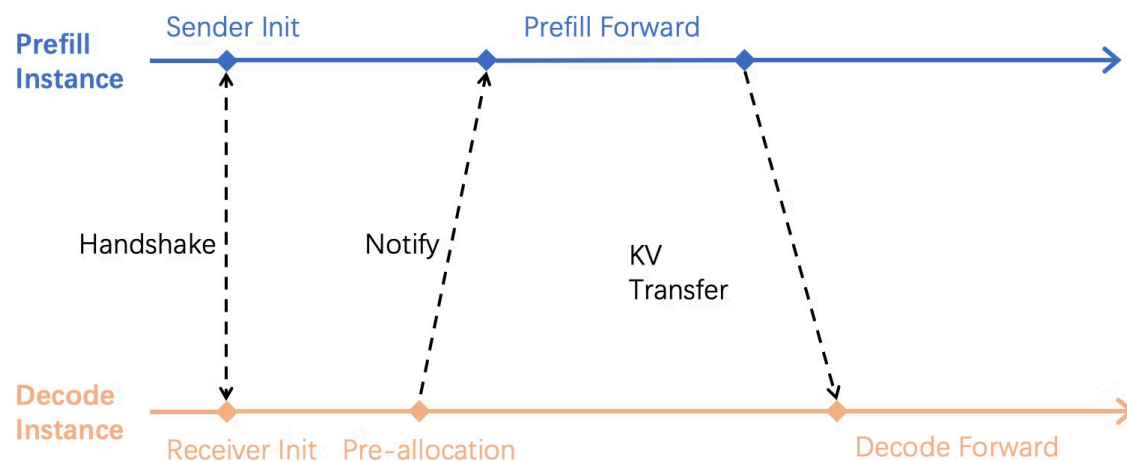
PD Disaggregation Architecture Design



- **Unified Load Balancer (LB)** for both **prefill** and **decode** paths. LB is **decoupled** from computation logic: requests are sent to LB and then routed to a selected PD pair.
- **Non-blocking** and **RDMA-based** KV cache transfer.
- **Flexible API Integration** for high-performance RDMA libraries like [Mooncake](#) and [NIXL](#) .

For more details: <https://github.com/sgl-project/sglang/issues/4655>

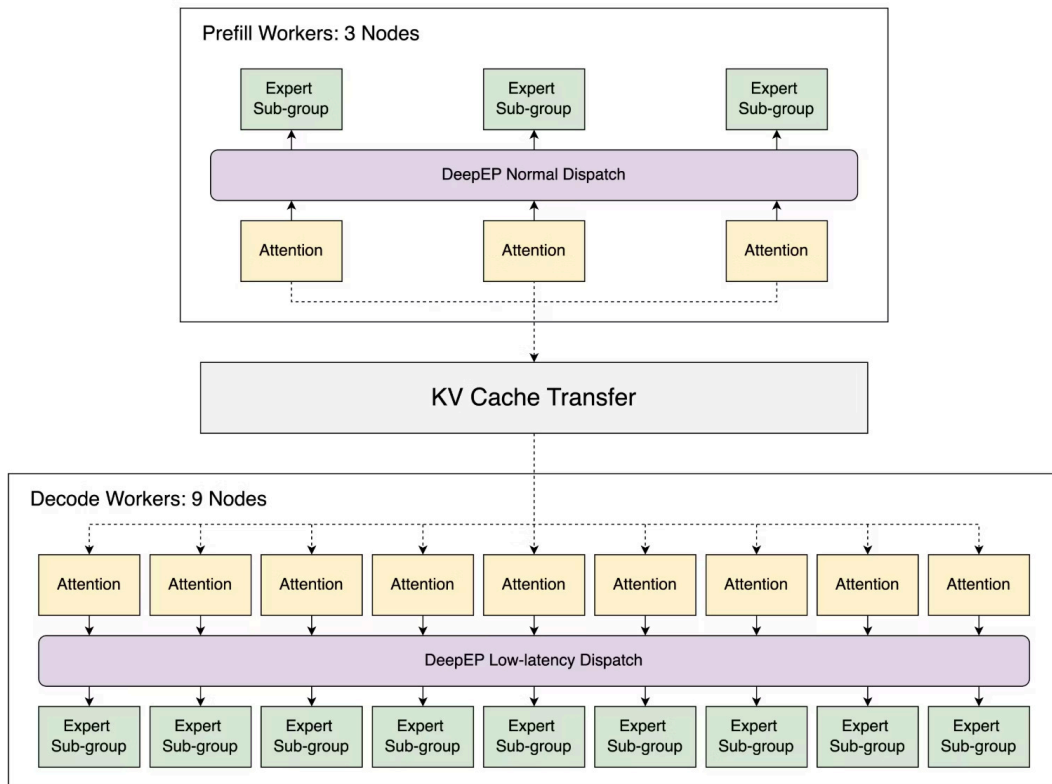
PD Disaggregation Timestamp



1. A Prefill Server and a Decode Server pair via a **handshake**, establishing a local sender and receiver respectively.
2. The Decode Server **pre-allocates** the KV cache, **signaling** the Prefill Server to begin the **model forward** pass and compute the KV caches.
3. Once computed, the **KV cache transfers** to the Decode Server, which handles iterative token generation.

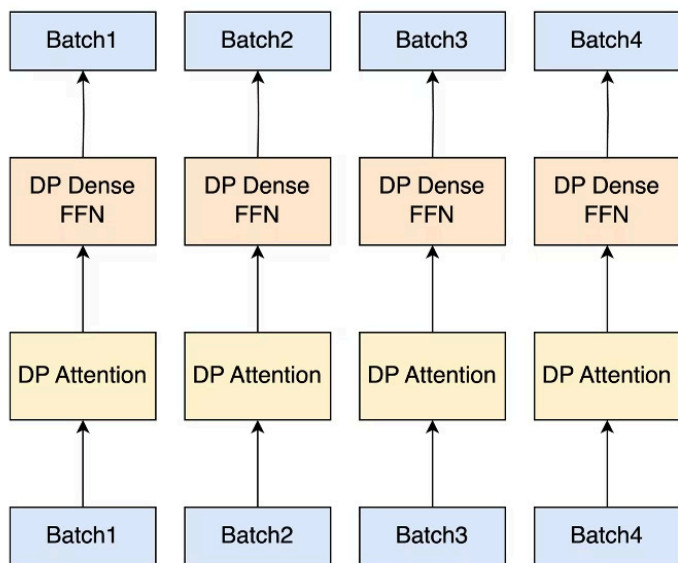
Large-scale Expert Parallelism

Large-scale EP with PD Disaggregation



- Deploy DeepSeek-V3 with **PD Disaggregation** and **large-scale EP** on **12 nodes * 8 GPUs**
- **52.3k** input tokens/s and **22.3k** output tokens/s per node for 2000-token input sequences
- This is the **first open-source implementation** to nearly match the throughput reported in the official DeepSeek blog

Parallelism Strategies with Attention & Dense FFN

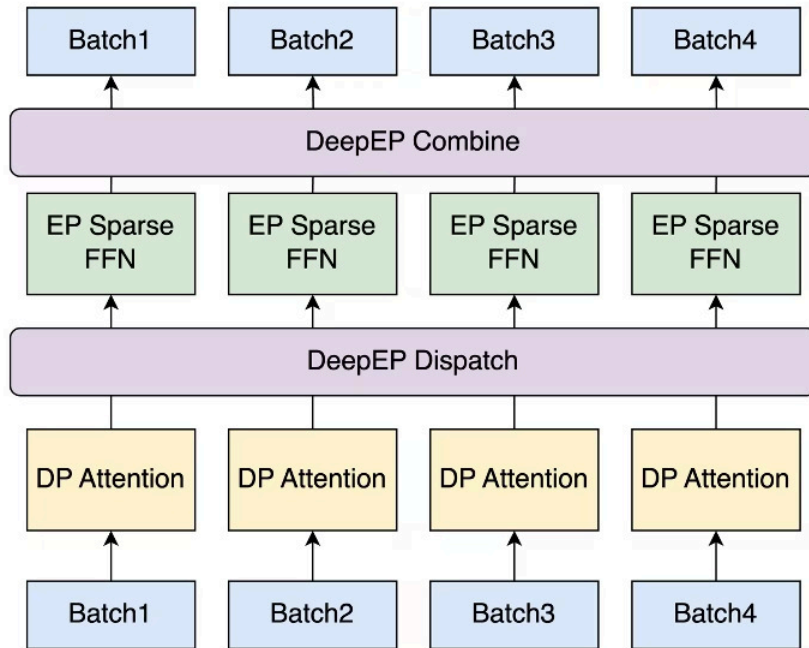


(a) DP Dense FFN with DP Attention

We use **Data Parallelism (DP)** instead of Tensor Parallelism (TP) for **Attention** and **Dense FFN**.

- **Optimized Memory Efficiency:** With **DP Attention**, the KV cache duplication across devices is eliminated, significantly reducing memory overhead.
- **Enhanced Scalability:** For Dense FFN intermediate dimension of 18,432, high TP degrees (e.g., TP32) result in inefficient fragmentation into small-unit segments (e.g., 576 units), which are **not divisible by 128** (Block size for Block-wise FP8 quantization).
- **Minimized Communication Overhead:** Replaces two all-reduces (in TP) with one reduce-scatter + one all-gather.

Parallelism Strategies with Sparse FFN (MoE)

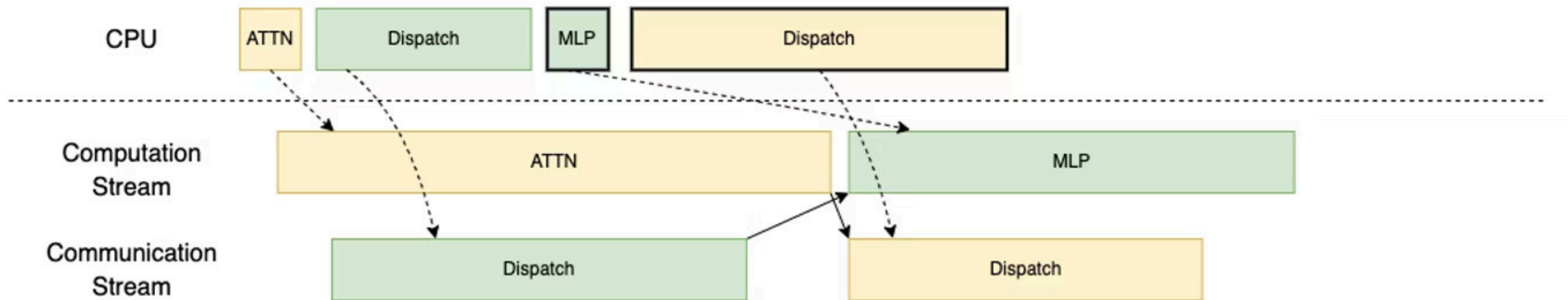


(b) EP Sparse FFN with DP Attention

We use **Expert Parallelism (DP)** for **Sparse FFN (MoE)**.

- **Scalable Model Capacity:** Expert weights are partitioned across devices using Expert Parallelism, removing memory bottlenecks.
- **Optimized Communication:** Follows a **Dispatch** → Expert → **Combine** pattern, powered by **DeepEP** and **Two-Batch-Overlap** to minimize latency and overhead.
- **Addressing Load Imbalance:** EP introduces variability in routing; EPLB and DeepEP optimize for workload distribution.

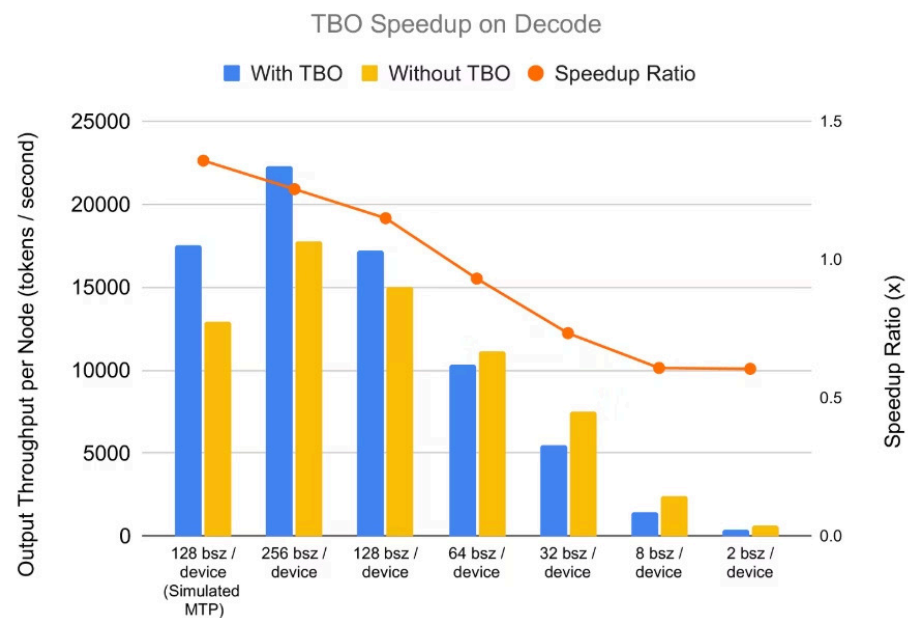
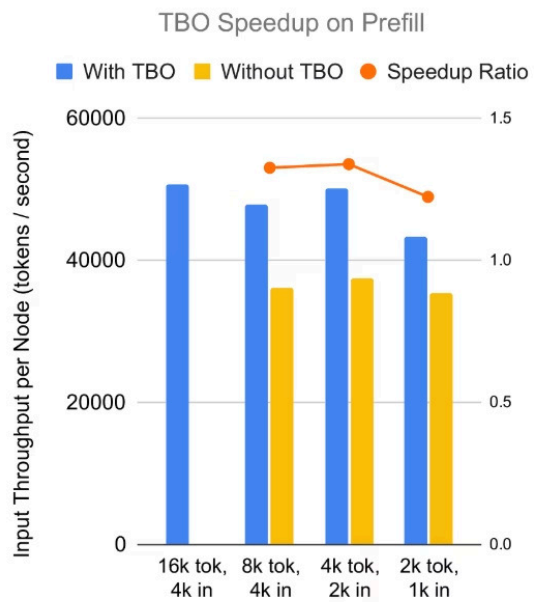
Two Batch Overlap



In multi-node environments, **limited communication bandwidth** can significantly increase overall latency.

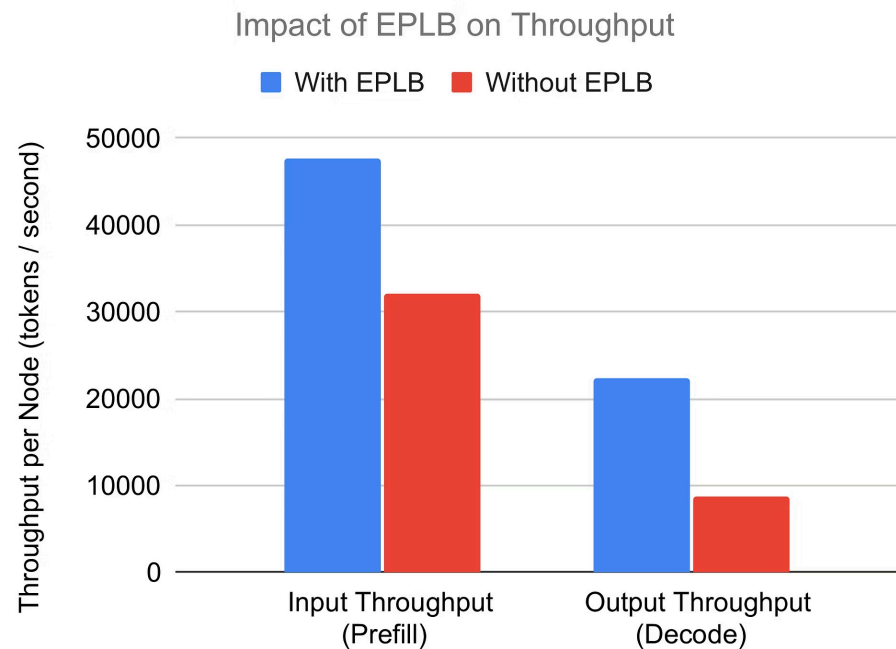
We implemented **Two-batch Overlap (TBO)** following [DeepSeek's system design](#). TBO splits a single batch into two micro-batches, allowing **computation and communication to overlap**, which also lowers peak memory usage by halving the effective batch size.

Two batch overlap



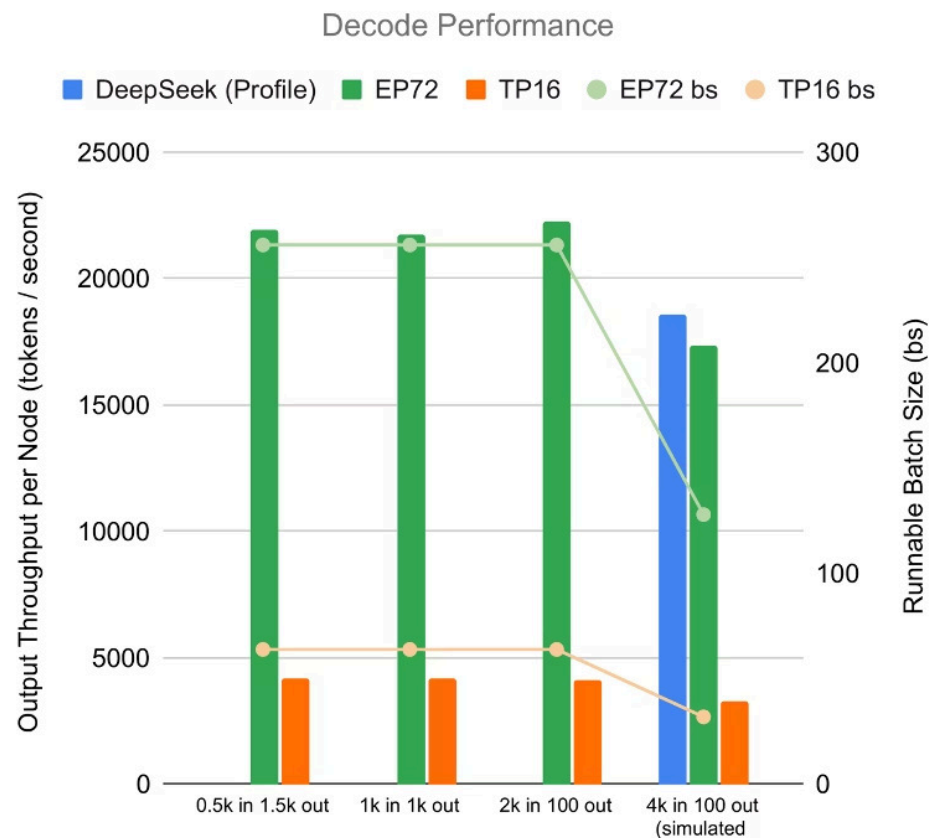
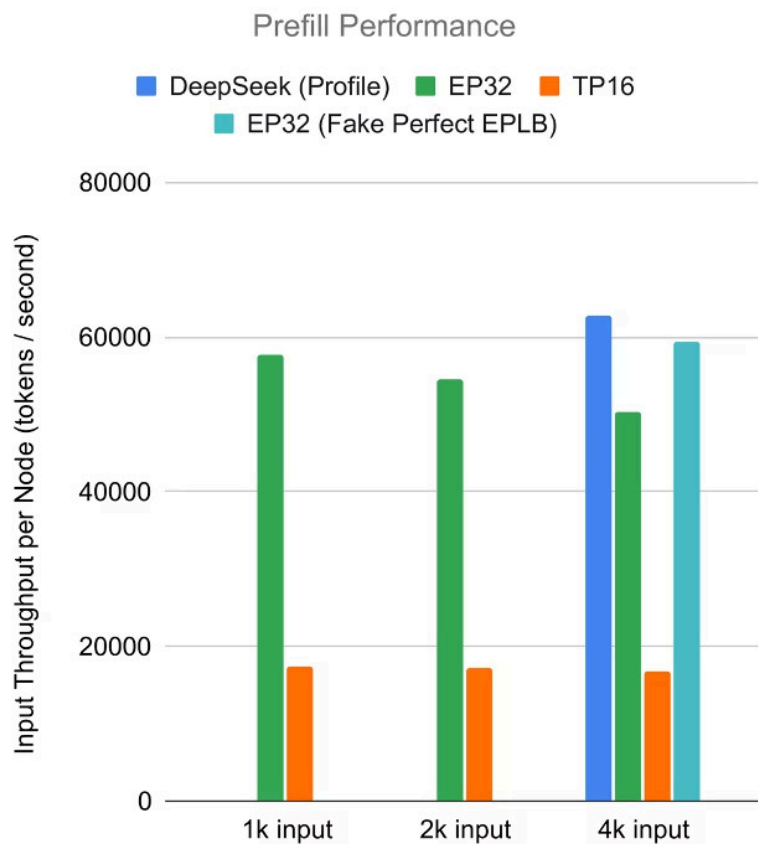
27% ~ 35% throughput increase for Prefill, and **25.5%** throughput increase at 256 batch size/device for Decode.

Expert Parallelism Load Balancer



- In MoE models, EP often leads to **uneven workload** distribution across GPUs. As the number of GPUs (EP size) increases, the imbalance issue gets more severe.
- **EPLB** takes **expert distribution statistics** as input and computes an optimal arrangement of experts to minimize imbalance.
- **EPLB** is supported in SGLang and delivers a significant speedup of **1.49x** (prefill) and **2.54x** (decode), due to its ability to mitigate workload imbalances across GPUs.

Benchmark Results



Throughputs of prefill (P) and decode (D) phases are evaluated independently, assuming unlimited resources for the non-tested phase to isolate and maximize the load on the tested nodes, mirroring the setup used by DeepSeek.

SGLang Ecosystem

Community adoptions



AMD



NVIDIA

intel

LinkedIn



CURSOR

ORACLE

Google Cloud

Microsoft
Azure

aws

Atlas Cloud



VOLTAGE PARK

NEBIUS



Novita



innomatrix



Ucla



Stanford



Berkeley
UNIVERSITY OF CALIFORNIA



baseten



Meituan



ANT
GROUP



Alibaba Cloud



Tencent Cloud

Community Links

- SGLang Slack: <https://slack.sglang.ai> (Welcome to join our Slack channel for further technical discussion!)
- Bi-Weekly Development Meeting: <https://meeting.sglang.ai>
- Documentation: <https://docs.sglang.ai>
- Blogs: <https://lmsys.org/blog>
- Learning Materials: <https://github.com/sgl-project/sgl-learning-materials>
- 2025 H1 Development Roadmap: <https://github.com/sgl-project/sglang/issues/4042>

Q&A